

Mini Vision Transformer 图像分类面试模板修改指南

适用于 ImageFolder 格式数据集：Data/train/class_x 与 Data/val/class_x

1. 模板适用场景

这份 MiniViT 模板适合图像分类任务，例如 ants/bees、cat/dog、normal/defect 等。它通过 torchvision.datasets.ImageFolder 自动从文件夹名称读取类别，并用轻量级 Vision Transformer 完成训练、验证、保存 checkpoint、绘制曲线和单张图片预测。

```
Data/
■■■ train/
■   ■■■ cat/
■   ■■■ dog/
■■■ val/
    ■■■ cat/
    ■■■ dog/
```

如果数据结构符合上面格式，你基本只需要改配置区，不需要改模型主体。

2. 面试现场首先检查数据结构

拿到数据集后，先确认根目录、train/val 是否存在、类别文件夹是否一致。

```
DATA_DIR = "Data"
print(os.listdir(DATA_DIR))
print(os.listdir(os.path.join(DATA_DIR, "train")))
print(os.listdir(os.path.join(DATA_DIR, "val")))
```

如果输出类似 [train, val]、[cat, dog]、[cat, dog]，说明模板可以直接读取。

3. 必须灵活修改的配置区

参数	面试中如何修改
DATA_DIR	改成面试给你的数据集路径，例如 "Data"。
OUTPUT_DIR	改成输出目录，例如 "outputs_mini_vit"。
IMG_SIZE	CPU 推荐 64；GPU 可用 128。图像越大，patch 越多，训练越慢。
PATCH_SIZE	CPU 推荐 8；GPU 可用 16。要求 IMG_SIZE 能被 PATCH_SIZE 整除。
IN_CHANNELS	RGB 图像用 3；灰度图用 1，并在 transforms 中加入 Grayscale。
D_MODEL	CPU 推荐 64；GPU 可用 128。必须能被 NHEAD 整除。
NHEAD	通常用 4。D_MODEL / NHEAD 必须是整数。
NUM_LAYERS	CPU 用 1；GPU 可用 2。层数越多越慢。
BATCH_SIZE	CPU 用 2 或 4；GPU 可用 8 或 16。
NUM_EPOCHS	面试演示用 1-3；有 GPU 可用 5。
NUM_WORKERS	Windows/Jupyter 建议 0；Linux 可尝试 2。

4. 推荐配置

CPU 面试演示配置：

```
DATA_DIR = "Data"
IMG_SIZE = 64
PATCH_SIZE = 8
IN_CHANNELS = 3
D_MODEL = 64
NHEAD = 4
NUM_LAYERS = 1
DIM_FEEDFORWARD = 128
BATCH_SIZE = 4
NUM_EPOCHS = 3
NUM_WORKERS = 0
LEARNING_RATE = 1e-3
```

GPU 面试演示配置：

```
IMG_SIZE = 128
PATCH_SIZE = 16
D_MODEL = 128
NHEAD = 4
NUM_LAYERS = 2
DIM_FEEDFORWARD = 256
BATCH_SIZE = 8
NUM_EPOCHS = 5
```

5. 为什么 IMG_SIZE 和 PATCH_SIZE 很重要？

Vision Transformer 的输入序列长度由 patch 数量决定。patch 数量越多，self-attention 计算越慢。

```
num_patches = (IMG_SIZE / PATCH_SIZE) * (IMG_SIZE / PATCH_SIZE)

# Example 1: IMG_SIZE=64, PATCH_SIZE=8
# num_patches = 8 * 8 = 64
# sequence length = 64 + 1 CLS token = 65

# Example 2: IMG_SIZE=224, PATCH_SIZE=8
# num_patches = 28 * 28 = 784
# sequence length = 785, much slower on CPU
```

面试中不要一开始就用 224x224 + patch size 8。CPU 上可能跑得很慢。

6. 数据加载部分如何工作？

模板中的 build_dataloaders() 使用 ImageFolder 自动读取类别。类别数量不需要手动写，来自 class_names = image_datasets["train"].classes。

```
image_datasets = {
    phase: datasets.ImageFolder(
        root=os.path.join(data_dir, phase),
        transform=data_transforms[phase]
    )
    for phase in ["train", "val"]
}

class_names = image_datasets["train"].classes
num_classes = len(class_names)
```

如果 Data/train 下有 cat、dog、rabbit 三个文件夹，模型输出维度会自动变成 3。

7. 模型主体通常不需要改

PatchEmbedding 使用 Conv2d 实现切 patch 和线性投影；MiniViT 使用 class token、positional embedding、TransformerEncoder

12. 面试时可以直接说的 plan

I will solve this as an image classification task using a lightweight Vision Transformer. First, I will inspect the dataset structure and check whether it follows the ImageFolder format with train and validation folders. Then I will resize all images, apply simple augmentation for training, and normalize them. Class names and number of classes will be inferred automatically from folder names.

For the model, I use a convolution-based patch embedding layer, then add a learnable class token and positional embedding. The token sequence is processed by a Transformer encoder, and the class token output is used for classification. Since this is an interview setting and may run on CPU, I use a small image size, small embedding dimension, and one Transformer encoder layer. Finally, I train with CrossEntropyLoss and AdamW, evaluate validation accuracy, save the best checkpoint, and visualize predictions.

13. 常见追问回答

为什么用 patch embedding ?

Transformer 需要序列输入，而图像是二维网格。patch embedding 把图像切成 patch，并把每个 patch 映射成 token。

为什么用 class token ?

class token 是可学习 token，经过 Transformer Encoder 后可以作为整张图像的全局表示。

为什么用 positional embedding ?

self-attention 本身不知道 patch 的空间顺序，positional embedding 提供位置信息。

为什么用 AdamW ?

AdamW 常用于 Transformer，优化更稳定，并且 decoupled weight decay 有助于正则化。

为什么不用 ResNet ?

ResNet transfer learning 对小数据集更稳，但这里是为了展示 Transformer-based vision pipeline。实际项目中可以比较 ResNet 和 MiniViT。

14. 最终面试修改清单

- DATA_DIR 改成 "Data" 或实际路径
- 确认数据是 ImageFolder 格式：train/val/class_name
- CPU 使用 IMG_SIZE=64, PATCH_SIZE=8, D_MODEL=64, NUM_LAYERS=1
- GPU 可提高到 IMG_SIZE=128, PATCH_SIZE=16, D_MODEL=128, NUM_LAYERS=2
- NUM_WORKERS 在 Windows/Jupyter 中设为 0
- RGB 图像 IN_CHANNELS=3；灰度图 IN_CHANNELS=1
- 类别数不手动写，使用 len(class_names)
- 单张预测只需要修改 test_img_path